Exercise Sheet 2

Information Security Lab: Module 2 • October 22, 2026

This exercise is **ungraded**.
Please think about these exercises *before* the session. We will discuss them together in class.

Exercise 1: Who Stops This Attack?

For each scenario, name the **adversary path** (software, DMA, physical) and the **mechanism** that is usually employed by TEEs to stop it (access control, encryption, integrity, freshness). Then say whether **SGX (client)** and **SEV-SNP** stop it.

1. The hypervisor maps an enclave/CVM page into its own address space and reads it.
2. An admin freezes a DIMM and reads it in another machine (cold boot).
3. A malicious device writes via DMA into a protected page.
4. A bus interposer records the ciphertext of a counter and writes it back later.
5. The hypervisor swaps the physical pages behind two guest-physical addresses of a CVM.
6. Code in a co-located VM hammers DRAM rows and flips a bit inside a protected page.

#	**Path**	**Mechanism needed**	**SGX (client)**	**SEV-SNP**
1				
2				
3				
4				
5				
6				

Exercise 2: The Heracles Byte-by-Byte Attack

Setting

A confidential VM's (CVM) memory is encrypted by the hardware in **4-byte blocks** (16 bytes in reality). A block with plaintext P at physical address $addr$ is stored as

$$C = E_K(P \oplus T(addr)) \oplus T(addr),$$

where E is a deterministic encryption algorithm, K is the secret encryption key and $T(addr)$ the tweak which depends only on the address and never changes.
The attacker is the malicious hypervisor. It does not know K and therefore *cannot* decrypt, but it can:

- **Read** the ciphertext of any CVM memory.
- **Move** a CVM page to an address it chooses. The page is automatically re-encrypted with the tweak of the new address, and the CVM does not notice.
- **Pause** the CVM at any point.
- **Inject** data it chooses into a CVM page (e.g., via network packets).

Task 1. Why does the tweak exist? If the same value is written to the *same* address twice, what do the two ciphertexts look like?

Task 2. A CVM stores a secret value S at address A on an encrypted block of a page. S uses all 4 bytes of the block. Use the hypervisor capabilities above to build a test with which the hypervisor can check: *"Is S equal to my guess g?"*.

Task 3. How many guesses does this test need in the worst case if S is completely unknown and the block size is 4 bytes? What about 16 bytes? And what if only **one byte** of S is unknown?

Task 4. A user types a password into a buffer at address A that starts as `00 00 00 00`. Each keystroke writes one character. The attacker pauses the VM after every keystroke. The alphabet is {`a`, `e`, `s`, `t`}. Each round, the attacker snapshots the block's ciphertext every time it changes and then tests guesses for the **new** character:

Round	Snapshot	new = `a`	new = `e`	new = `s`	new = `t`
1	`9E07C4A1`	`41D28F3B`	`7AB31E96`	`E610D25F`	`9E07C4A1`
2	`2C58A70E`	`D90F6B24`	`2C58A70E`	`B7A1093C`	`6E3CF581`
3	`F4135D8B`	`F4135D8B`	`0B9DE247`	`57E2B6A0`	`A86C3F19`

How can the hypervisor recover the password? How many tries does it need in the worst case to recover the password?

Task 5. What happens if the attacker misses a keystroke? Name one defense that stops the attack.

Exercise 3: DMA Attacks

Setting

Peripherals such as network cards (NICs) and GPUs use **DMA** (Direct Memory Access): they read and write main memory directly, without going through the CPU. To limit what a device can touch, the system has an **IOMMU**:

- The hypervisor manages an **I/O page table** for each device. The table maps device addresses (IOVAs) to physical pages, with read/write permissions.
- The IOMMU checks every DMA request against the device's I/O page table.

Checking every DMA against the IOMMU page table is slow. PCIe therefore offers **ATS** (Address Translation Services):

- A device asks the IOMMU for a translation (IOVA $\rightarrow$ physical address) and caches the answer in its **own on-device translation cache (IOTLB)**.
- From then on, the device sends requests that are marked **"already translated"** and use the physical address. The IOMMU lets these through without checking the page table again.
- When the hypervisor unmaps a page, it removes the entry in the I/O page-table and **immediately** sends the device an **invalidation request** ("drop your translation for this IOVA"). The device answers with an **invalidation completion**.

Attacker: a malicious device plugged into a PCIe port. The device can send any DMA request it likes and ignore any protocol rules.

Task 1. We assume the hypervisor, which configures the IOMMU, is trusted in this task. The device reports that it supports ATS, and the hypervisor turns ATS on.

(a) The hypervisor has freed a page P that was previously used by the device, which is now reused elsewhere in the kernel. List every place where a translation for P existed before the unmap. Which of them does the hypervisor control directly, and which does it not?

(b) The ATS invalidation request was sent to the device when the page P was freed. Is the device now guaranteed to have lost access to P?

(c) Read the rules for "already translated" requests again. Does a malicious device actually need a real translation to access a page?

(d) How would you fix the attack while keeping the device usable?

Task 2. Now think of a confidential VM (CVM) with a passed-through device and an untrusted hypervisor. Who configures the IOMMU and decides whether ATS is enabled? Can the CVM rely on the IOMMU to keep the device away from its memory? What would you need instead?